

Università degli Studi di Padova

DIPARTIMENTO DI MATEMATICA “TULLIO LEVI-CIVITA”

CORSO DI LAUREA IN INFORMATICA



Explainable AI per la pianificazione della
produzione in un gestionale

Tesi di Laurea

Relatore

Prof. Zanella Marco

Laureando

Simonetto Federico

Matricola 2113180

ANNO ACCADEMICO 2025-2026

“Success is the sum of small efforts, repeated day in and day out.”

— Robert Collier

Ringraziamenti

Desidero esprimere la mia gratitudine al professor Zanella Marco, mio relatore, per l'aiuto e il sostegno che mi ha dato durante la stesura dell'elaborato.

Vorrei anche ringraziare, con affetto, i miei genitori per il loro sostegno, il grande aiuto e la loro presenza in ogni momento durante gli anni di studio.

Desidero poi ringraziare i miei amici per i bellissimi anni trascorsi insieme e le mille avventure vissute.

Padova, Settembre 2026

Simonetto Federico

Sommario

Il presente elaborato descrive l'integrazione di un sistema di Explainable Artificial Intelligence (XAI) all'interno del sistema Enterprise Resource Planning (ERP) sviluppato da Spazio Dev S.r.l. L'obiettivo del sistema XAI è fornire una spiegazione comprensibile e verificabile delle decisioni prese dal gestionale, attraverso l'analisi delle decisioni generate, delle evidenze associate e degli snapshot dei dati utilizzati nel processo decisionale.

All'interno dell'elaborato vengono descritte le principali attività svolte durante il progetto, comprendenti l'analisi dei requisiti del sistema, la definizione delle scelte architetturali, la fase di implementazione, le strategie di testing adottate e le considerazioni conclusive sui risultati ottenuti.

Indice

Glossario	xi
1 Introduzione	1
1.1 L'azienda	1
1.1.1 Il reparto	1
1.2 L'idea	2
1.3 Organizzazione del testo	3
2 Processi e metodologie	5
2.1 Contesto aziendale	5
2.1.1 Modello di sviluppo Agile	5
2.1.2 Issue Tracking System	6
2.1.3 Version Control	7
2.1.4 Il progetto	8
2.1.5 Pianificazione delle attività	9
2.1.6 Modalità di lavoro e collaborazione	10
2.2 Obiettivi dello stage	10
2.3 Analisi preventiva dei rischi	12
2.3.1 Difficoltà di integrazione con componenti esistenti	12
2.3.2 Incompatibilità con il solver di scheduling	12
2.3.3 Elevata latenza nei tempi di risposta del sistema	13
2.3.4 Indisponibilità o malfunzionamento delle API esterne	13
3 Analisi dei requisiti	15
3.1 Tipologie di utenti	15

3.2	Casi d'uso	16
3.2.1	Struttura dei casi d'uso	16
3.3	Elenco casi d'uso	17
3.3.1	UC1: Configurazione del modulo XAI	17
3.3.1.1	UC1.1: Selezione del modello LLM per il modulo XAI	18
3.3.1.2	UC1.2: Configurazione degli utenti abilitati al modulo XAI	18
3.3.1.3	UC1.3: Configurazione dei tool disponibili per il modulo XAI	19
3.3.2	UC2: Richiesta di spiegazioni al modulo XAI su una de- cisione del sistema	19
3.3.2.1	UC2.1: Gestione della conversazione con il mo- dulo XAI	20
3.3.2.2	UC2.2: Chiamata a tool per spiegazioni del mo- dulo XAI	21
3.3.2.3	UC2.3: Modello LLM non raggiungibile	21
3.3.2.4	UC2.4: Limite token di risposta superato	22
3.3.2.5	UC2.5: Permessi non sufficienti per effettuare la richiesta	22
3.3.3	UC3: Richiesta creazione nuovi record	23
3.3.3.1	UC3.1: Chiamata al tool per creazione record del modulo XAI	24
3.3.3.2	UC3.2: Dati forniti incorretti per la creazione del record	25
3.3.4	UC4: Configurazione delle regole alert	25
3.3.5	UC5: Visualizzazione delle notifiche alert	26
3.3.5.1	UC5.1: Numero elevato di violazioni rilevate	26
3.3.6	UC6: Visualizzazione del dettaglio dell'alert	27
3.3.7	UC7: Visualizzazione del modello associato dell'alert	27
3.3.8	UC8: Richiesta al modulo XAI di una spiegazione relativa all'alert generato	28

3.3.9	UC9: Visualizzazione log operazioni	29
3.3.10	UC10: Visualizzazione dei dettagli di un'operazione	30
3.4	Tracciamento dei requisiti	31
3.4.1	Requisiti funzionali	32
3.4.2	Requisiti di qualità	34
3.4.3	Requisiti di vincolo	35
4	Progettazione e codifica	36
4.1	Tecnologie e strumenti	36
4.1.1	Linguaggi di programmazione	36
4.1.2	Framework e librerie	36
4.1.2.1	Laravel	36
4.1.2.2	Filament	37
4.1.2.3	Livewire	37
4.1.3	Tecnologie di persistenza e gestione dei dati	37
4.1.3.1	MariaDB	37
4.1.3.2	Redis	37
4.1.3.3	Laravel Horizon	38
4.1.4	Ambiente di sviluppo e containerizzazione	38
4.1.4.1	Docker	38
4.1.4.2	DDEV	38
4.1.5	Strumenti di qualità e testing	39
4.1.5.1	Pest	39
4.1.5.2	PHPStan e Larastan	39
4.2	Architettura	39
4.3	Struttura del progetto	41
4.4	Componenti principali	42
4.4.1	Modulo XAI	42
4.4.1.1	Tool	42
4.4.1.1.1	FinderTool	42
4.4.1.2	Provider	43
4.4.1.2.1	RunpodOllamaProvider	44

4.4.1.3	Services	45
4.4.1.3.1	AiChatService	45
4.4.2	Sistema di generazione alert	46
4.4.2.1	Rules	46
4.4.3	Sistema di generazione degli alert	46
4.4.3.1	Rules	47
4.4.3.2	Jobs	48
4.4.3.2.1	AlertJob	48
4.4.3.2.2	BroadcastAlertJob	49
4.4.3.3	Command	49
4.4.3.3.1	AlertCommand	49
4.4.3.4	Resources e Pages	49
4.4.4	Sistema di logging delle operazioni	50
4.4.4.1	Observers	50
4.4.4.1.1	AuditObserver	50
4.4.4.2	Resources e Pages	51
4.5	Design Pattern utilizzati	51
4.5.1	Observer Pattern	51
4.5.2	Strategy Pattern	52
4.5.3	Command Pattern	52
4.5.4	Template Method Pattern	53
Bibliografia		i
Sitografia		ii

Elenco delle figure

1.1	Logo dell'azienda	1
2.1	Modello Scrum	6
2.2	Modello GitFlow	8

Elenco delle tabelle

2.1	Tabella riguardante i requisiti dell'attività di stage	11
3.2	Tabella riguardante i requisiti funzionali del progetto	33
3.3	Tabella riguardante i requisiti di qualità del progetto	34
3.4	Tabella riguardante i requisiti di vincolo del progetto	35

Elenco dei codici sorgenti

4.1	Struttura del progetto	41
-----	----------------------------------	--------------------

Glossario

AI Disciplina dell'informatica che si occupa dello sviluppo di sistemi capaci di eseguire attività che normalmente richiedono capacità tipiche dell'intelligenza umana, come apprendimento, ragionamento e presa di decisioni.

[4](#)

APS Sistema di pianificazione avanzata della produzione che permette di ottimizzare l'allocazione delle risorse produttive, definendo tempi e sequenze operative in base alla capacità disponibile e alla domanda prevista. [3](#)

ATP Sistema di pianificazione avanzata della produzione che permette di ottimizzare l'allocazione delle risorse produttive, definendo tempi e sequenze operative in base alla capacità disponibile e alla domanda prevista. [3](#)

DTO design pattern utilizzato per raggruppare e trasportare dati tra diverse parti o livelli di un'applicazione (ad esempio da un database al backend, o dal backend a un client/frontend) tramite un'unica chiamata o struttura, senza includere alcuna logica di business.. [46](#), [47](#)

Forecast Previsione della domanda futura ottenuta mediante l'analisi di dati storici e informazioni disponibili, utilizzata per supportare la pianificazione aziendale.. [2](#)

Git Sistema di controllo delle versioni (VCS) open-source e gratuito che consente di gestire le modifiche ai codici sorgenti, creando una storia di tutte le versioni del progetto.. [7](#)

LLM algoritmo di intelligenza artificiale basato sul deep learning e sull'architettura Transformer, progettato per comprendere, elaborare e generare testo in linguaggio naturale in modo simile a un essere umano. [17](#)

MRP Metodo di pianificazione della produzione che consente di determinare quantità e tempi di approvvigionamento delle materie prime necessarie per soddisfare la domanda prevista, considerando disponibilità di magazzino e ordini pianificati. [2](#)

Snapshot Copia dello stato di un insieme di dati in un determinato istante temporale, utilizzata per conservare le informazioni disponibili al momento di una decisione o di un evento.. [3](#)

UML linguaggio di modellazione visuale utilizzato per rappresentare, specificare e documentare la struttura e il comportamento di un sistema software. Attraverso una serie di diagrammi standardizzati, UML permette di descrivere diversi aspetti del sistema, come le componenti che lo costituiscono, le relazioni tra gli elementi e le interazioni tra gli utenti e le funzionalità offerte.. [16](#)

XAI Insieme di metodi e tecniche che permettono di rendere comprensibili e interpretabili agli utenti umani i risultati prodotti da modelli di intelligenza artificiale, favorendone la fiducia e la verifica. [2](#)

Capitolo 1

Introduzione

1.1 L'azienda

Spazio Dev S.r.l.¹ è un'azienda multidisciplinare con sede a Tombolo (Padova) specializzata nello sviluppo di soluzioni software innovative. Integrando tecnologie avanzate, intelligenza artificiale e design, trasforma idee ed esigenze aziendali in prodotti pronti per il mercato. Dal 2023 supporta imprese e realtà produttive nella realizzazione di software su misura, promuovendo innovazione, efficienza e competitività attraverso l'impiego delle più moderne tecnologie.



Figura 1.1: Logo dell'azienda

1.1.1 Il reparto

Dev-Automation è la divisione dell'azienda specializzata nello sviluppo e nell'integrazione di soluzioni basate sull'Intelligenza Artificiale per il settore industriale. L'obiettivo del reparto è supportare le aziende nell'ottimizzazione dei processi produttivi attraverso strumenti intelligenti e tecnologie innovative. I prodotti da loro sviluppati comprendono:

¹<https://www.spaziodev.eu>

- **Brain/ErpAI:** piattaforma ERP potenziata dall'Intelligenza Artificiale che supporta la pianificazione della produzione, la gestione delle risorse e l'ottimizzazione dei processi aziendali. Il progetto descritto nel presente documento è stato sviluppato all'interno di questo prodotto.
- **Fault Prevention:** modulo dedicato al monitoraggio continuo della produzione, progettato per prevenire i fermi macchina mediante l'analisi in tempo reale dei dati provenienti dagli impianti.
- **Soluzioni Custom:** progettazione e sviluppo di sistemi basati sull'Intelligenza Artificiale realizzati su misura, in grado di rispondere alle specifiche esigenze operative e produttive delle aziende clienti.

1.2 L'idea

Il progetto *Brain* è sviluppato su commissione di Falmec S.p.A., azienda specializzata nella progettazione e produzione di cappe aspiranti per cucine domestiche. L'obiettivo del progetto è ottimizzare l'intero processo di gestione aziendale, coprendo tutte le principali aree operative: dalla gestione del magazzino alla pianificazione della produzione, fino alla gestione delle vendite.

Per raggiungere tale obiettivo, *Brain* implementa un sistema gestionale basato sulla metodologia *Material Requirements Planning (MRP)_G*. Attraverso l'analisi dello storico delle vendite e dei *Forecast_G* sulla domanda futura, il sistema consente di adottare un approccio proattivo alla pianificazione della produzione. In questo modo, l'azienda è in grado di organizzare con anticipo le risorse necessarie, anziché intervenire solamente al momento della ricezione degli ordini, migliorando l'efficienza operativa e riducendo tempi di risposta e sprechi.

L'attività di stage si inserisce in questo contesto con l'obiettivo di integrare all'interno di *Brain* un sistema di *Explainable Artificial Intelligence (XAI)_G* auditabile e un sistema di gestione degli alert.

Il modulo XAI dovrà consentire la registrazione e la consultazione delle decisioni prodotte dai diversi moduli del gestionale, memorizzando per ciascuna di esse le

evidenze, i fattori che hanno influenzato il processo decisionale e uno *Snapshot_G* dei dati disponibili al momento della decisione.

Parallelamente, verrà sviluppato un motore di alerting in grado di individuare e segnalare situazioni critiche all'interno dei processi produttivi, fornendo agli utenti informazioni tempestive sulle eventuali anomalie rilevate. Il sistema dovrà inoltre permettere la definizione di regole personalizzabili, così da adattare la generazione degli alert alle specifiche esigenze operative degli utenti finali.

L'integrazione interesserà in particolare i moduli *Artificial Intelligence (APS)_G*, *Available To Promise (ATP)_G* e il calcolo del costo effettivo, nei quali vengono generate decisioni o risultati di particolare rilevanza. Tali componenti verranno collegati al modulo XAI affinché ogni decisione possa essere resa comprensibile, tracciabile e verificabile. Inoltre, il sistema consentirà di individuare rapidamente eventuali scostamenti rispetto ai valori attesi o situazioni critiche, notificandole agli utenti mediante un sistema di alert configurabili e facilitando così interventi tempestivi.

1.3 Organizzazione del testo

Il secondo capitolo: "Processi aziendali e metodologie di lavoro" descrive i processi aziendali e le metodologie di lavoro utilizzate durante il periodo di stage, fornendo una panoramica completa delle attività svolte.

Il terzo capitolo: "Analisi dei requisiti del progetto" analizza i requisiti del progetto e le loro relazioni con le funzionalità del sistema, utilizzando metodi di analisi avanzati per garantire la qualità del risultato.

Il quarto capitolo: "Progettazione e codifica del sistema" descrive la progettazione e la codifica del sistema, fornendo dettagli sulla scelta delle tecnologie e sulle strategie di sviluppo utilizzate.

Il quinto capitolo: "Verifica e valutazione delle prestazioni del sistema" verifica e valuta le prestazioni del sistema, utilizzando metodi di testing avanzati per garantire la qualità e la sicurezza del risultato.

Il sesto capitolo: "Conclusione e prospettive future" descrive le conclusioni e le future prospettive del progetto, fornendo una panoramica completa delle attività svolte e dei risultati ottenuti.

Riguardo la stesura del testo, relativamente al documento sono state adottate le seguenti convenzioni tipografiche:

- gli acronimi, le abbreviazioni e i termini ambigui o di uso non comune menzionati vengono definiti nel glossario, situato all’inizio del presente documento;
- per la prima occorrenza dei termini riportati nel glossario viene utilizzata la seguente nomenclatura: AI_G ;
- i termini in lingua straniera o facenti parti del gergo tecnico sono evidenziati con il carattere *corsivo*.

Capitolo 2

Processi e metodologie

2.1 Contesto aziendale

L'azienda si distingue per l'elevata attenzione rivolta all'organizzazione del lavoro e alla definizione dei processi di sviluppo, aspetti che hanno contribuito a creare un ambiente collaborativo ed efficiente. Grazie a una gestione strutturata delle attività, il team di sviluppo del progetto *ErpAI* ha potuto operare in maniera coordinata, favorendo una comunicazione efficace e una rapida risoluzione delle problematiche emerse durante lo sviluppo.

Il reparto dedicato al progetto *ErpAI* è composto da quattro sviluppatori, incluso il sottoscritto, coordinati dal titolare dell'azienda, che ha ricoperto un ruolo di supervisione, fornendo supporto tecnico e indicazioni progettuali durante l'intero periodo di sviluppo.

2.1.1 Modello di sviluppo Agile

L'azienda adotta il framework *Scrum*¹ come metodologia di sviluppo software, organizzando il lavoro in sprint della durata di una settimana.

Ogni sprint ha inizio con lo *Sprint Planning*, incontro coordinato dallo *Scrum Master*, figura responsabile del corretto svolgimento del processo Scrum. Durante questa fase vengono definite le funzionalità da sviluppare nel corso della settimana e vengono analizzate le criticità emerse durante la *Sprint Retrospective*

¹*Scrum*. URL: <https://scrumguides.org/scrum-guide.html>

dello sprint precedente, con l'obiettivo di migliorare continuamente il processo di sviluppo.

Nel corso dello sprint, il team lavora in modo autonomo al raggiungimento degli obiettivi prefissati. Al termine della settimana si svolge la *Sprint Review*, durante la quale vengono presentati i risultati ottenuti e raccolti i feedback necessari per valutare il lavoro svolto e pianificare le attività successive. La Figura 2.1 riassume schematicamente il ciclo iterativo seguito durante lo sviluppo.

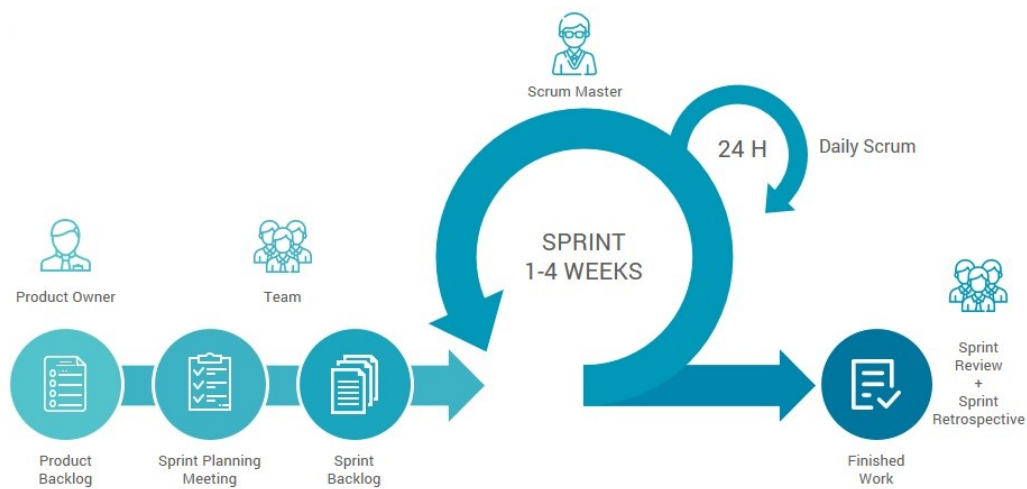


Figura 2.1: Modello Scrum

2.1.2 Issue Tracking System

La gestione delle attività di sviluppo è affidata a Plane², una piattaforma di *issue tracking* che consente di organizzare e monitorare l'avanzamento del progetto attraverso una bacheca suddivisa nelle seguenti sezioni:

- **Backlog:** raccoglie le attività pianificate ma non ancora avviate;
- **To Do:** contiene le attività pronte per essere sviluppate;
- **In Progress:** comprende le attività attualmente in fase di sviluppo, revisione o testing;

²Plane. URL: <https://plane.so/>

- **Done:** raccoglie le attività completate e chiuse.

Ogni issue contiene una serie di informazioni utili alla gestione del lavoro, tra cui:

- **Titolo:** breve descrizione dell'attività da svolgere;
- **Descrizione:** dettagli relativi ai requisiti e agli obiettivi dell'attività;
- **Assegnatari:** sviluppatori responsabili della realizzazione dell'issue;
- **Stato:** indica la fase del ciclo di sviluppo in cui si trova l'attività.

2.1.3 Version Control

La gestione del codice sorgente è affidata al sistema di controllo di versione *Git*³, utilizzato tramite Gitea³, una piattaforma open-source per l'hosting di repository Git all'interno dell'infrastruttura aziendale.

L'organizzazione dei branch si ispira al modello GitFlow (illustrato in Figura 2.2), opportunamente adattato alle esigenze del team di sviluppo. In particolare, vengono utilizzate le seguenti tipologie di branch:

- **main:** contiene la versione stabile del software destinata alla produzione;
- **dev:** rappresenta il ramo principale di integrazione delle funzionalità in fase di sviluppo;
- **feature/<numero-issue>:** utilizzato per lo sviluppo di nuove funzionalità, identificate tramite il codice univoco dell'issue corrispondente;
- **fix:** dedicato alla correzione di bug o anomalie riscontrate durante lo sviluppo;
- **release:** utilizzato per preparare una nuova versione del software prima del rilascio.

³Gitea. URL: <https://about.gitea.com/>



La revisione da parte degli altri membri del team consente di individuare eventuali problematiche, verificare la conformità agli standard adottati e garantire una maggiore qualità del codice prima del suo inserimento nei rami condivisi del progetto.

L'attività di stage si è concentrata sullo sviluppo di due componenti principali del sistema:

- **Modulo XAI:** progettazione e integrazione di un chatbot auditabile all'interno del gestionale, in grado di fornire spiegazioni sulle decisioni prese dal sistema. Il modulo consente inoltre di registrare tali spiegazioni e renderle successivamente consultabili tramite un sistema di storico (*history*), garantendo tracciabilità e auditabilità delle decisioni.
- **Sistema di alert:** progettazione e sviluppo di un motore di gestione degli alert finalizzato all'individuazione di criticità nei processi produttivi. Il sistema permette inoltre agli utenti finali di definire e personalizzare regole di generazione degli alert in funzione delle specifiche esigenze operative.

Un ulteriore obiettivo dello stage consiste nell'integrare all'interno delle due componenti sviluppate alcune funzionalità introdotte nella terza consegna del progetto, in particolare un *solver di scheduling*, responsabile della risoluzione di problemi di ottimizzazione relativi alla pianificazione delle attività e all'allocazione delle risorse disponibili. L'attività si limita tuttavia a integrazioni di carattere leggero, finalizzate a garantire la compatibilità tra il nuovo componente e le funzionalità sviluppate durante lo stage.

2.1.5 Pianificazione delle attività

Lo stage si è sviluppato nell'arco di 300 ore complessive, distribuite su sette settimane lavorative da 40 ore ciascuna e un'ultima settimana conclusiva da 20 ore.

Le prime due settimane sono state dedicate all'inserimento nel contesto progettuale, attraverso lo studio delle tecnologie adottate dall'azienda, necessarie per lo sviluppo delle componenti previste, l'analisi del lavoro precedentemente svolto dagli altri membri del team e la definizione dei requisiti funzionali che le componenti realizzate durante lo stage avrebbero dovuto soddisfare.

Le settimane successive sono state principalmente dedicate all'attività di sviluppo e integrazione delle funzionalità previste dal progetto. Le ultime 40 ore sono state invece destinate alle attività di testing, alla verifica del corretto funzionamento delle componenti implementate e alla produzione della documentazione

tecnica relativa al lavoro svolto, con l'obiettivo di garantire la manutenibilità, l'estendibilità e l'auditabilità del codice prodotto.

2.1.6 Modalità di lavoro e collaborazione

L'attività di tirocinio è stata svolta interamente in presenza, favorendo un confronto diretto e continuo con il tutor aziendale e con gli altri sviluppatori coinvolti nel progetto *ErpAI*.

Oltre agli incontri quotidiani dedicati all'allineamento sulle attività svolte e sulle problematiche emerse, il team ha utilizzato strumenti di comunicazione asincrona per la gestione di dubbi o criticità di minore entità. Questo ha permesso di ottenere rapidamente chiarimenti e feedback, mantenendo un flusso di collaborazione costante anche al di fuori degli incontri programmati.

2.2 Obiettivi dello stage

Gli obiettivi dello stage sono stati definiti in accordo con il tutor aziendale e classificati in base alla loro priorità. Gli obiettivi **obbligatori** rappresentano le funzionalità essenziali da realizzare per il completamento del progetto, quelli **desiderabili** costituiscono miglioramenti che incrementano il valore del sistema senza essere indispensabili, mentre gli obiettivi **funzionali** individuano possibili estensioni e integrazioni da sviluppare qualora il tempo a disposizione lo consenta.

Codice	Descrizione requisito	Tipologia
O01	Implementare un sistema XAI auditabile per registrare e consultare decisioni, fattori, evidenze e snapshot dei dati.	Obbligatorio
O02	Realizzare regole alert, eventi e notifiche configurabili per segnalare proattivamente criticità operative.	Obbligatorio
O03	Integrare i punti decisionali della terza consegna nei log XAI e negli alert, limitandosi a controlli, riprese in mano e integrazioni leggere.	Obbligatorio
D01	Differenziare le spiegazioni XAI e i dettagli visibili in base al ruolo dell'utente.	Desiderabile
D02	Predisporre run di ottimizzazione tracciabili con input, output, score, metriche e log decisionale associato.	Desiderabile
D03	Introdurre dashboard o riepiloghi sugli alert generati, presi in carico e risolti.	Desiderabile
F01	Collegare un solver o microservizio esterno per scenari specialistici come nesting laser o sequenziamento avanzato.	Funzionale
F02	Aggiungere suggerimenti automatici di configurazione soglie basati sui dati storici.	Funzionale
F03	Integrare le spiegazioni XAI nel chatbot sviluppato da un altro piano di lavoro o modulo parallelo.	Funzionale

Tabella 2.1: Tabella riguardante i requisiti dell'attività di stage

2.3 Analisi preventiva dei rischi

Durante la fase iniziale dello stage sono stati identificati alcuni possibili rischi che avrebbero potuto influenzare il corretto svolgimento delle attività previste. Per ciascun rischio sono state individuate probabilità che il rischio si presenti così come opportune strategie di mitigazione volte a ridurre l'impatto sul progetto.

2.3.1 Difficoltà di integrazione con componenti esistenti

Probabilità: Media.

Impatto: Alto.

Descrizione: Le nuove funzionalità sviluppate durante lo stage devono essere integrate all'interno di un gestionale già esistente e caratterizzato da un'architettura complessa. Una conoscenza inizialmente limitata della struttura del software e delle interazioni tra i diversi moduli potrebbe causare errori di integrazione, malfunzionamenti o un aumento dei tempi di sviluppo. L'assenza di una documentazione completa e aggiornata dell'architettura esistente aumenta significativamente l'impatto di questo rischio, poiché rende più difficile comprendere il funzionamento del sistema e individuare correttamente i punti di integrazione delle nuove funzionalità.

Soluzione: Per mitigare tale rischio è stata prevista una fase iniziale di analisi dell'architettura del sistema e del codice esistente, accompagnata da un confronto continuo con il tutor aziendale e con gli sviluppatori che avevano realizzato le componenti già presenti.

2.3.2 Incompatibilità con il solver di scheduling

Probabilità: Bassa.

Impatto: Medio.

Descrizione: L'integrazione con il solver di scheduling sviluppato nelle precedenti consegne potrebbe introdurre problematiche di compatibilità tra le componenti, dovute a differenze nelle interfacce o nelle modalità di gestione dei dati.

Soluzione: L'integrazione è stata pianificata attraverso un'analisi preventiva delle interfacce esposte dal solver e una definizione delle modalità di comunicazione necessarie, limitando l'intervento alle modifiche strettamente necessarie.

2.3.3 Elevata latenza nei tempi di risposta del sistema

Probabilità: Media.

Impatto: Medio.

Descrizione: L'integrazione di funzionalità basate su intelligenza artificiale, come il chatbot XAI, può introdurre un aumento dei tempi di risposta rispetto alle funzionalità tradizionali del gestionale. Tempi di elaborazione elevati potrebbero compromettere l'esperienza dell'utente e rendere meno efficace l'interazione con il sistema.

Soluzione: È stata prevista un'analisi delle tempistiche di risposta delle componenti coinvolte, valutando eventuali ottimizzazioni nelle modalità di comunicazione e nella gestione delle richieste. Inoltre, sono state considerate strategie per migliorare la percezione dell'attesa da parte dell'utente, come la gestione asincrona delle operazioni più onerose.

2.3.4 Indisponibilità o malfunzionamento delle API esterne

Probabilità: Bassa.

Impatto: Alto.

Descrizione: Il modulo XAI dipende dall'utilizzo di API esterne per l'elaborazione delle richieste. Eventuali indisponibilità del servizio, errori di comunicazione, modifiche alle interfacce o un aumento della latenza potrebbero compromettere il corretto funzionamento del chatbot e delle funzionalità ad esso collegate.

Soluzione: Per ridurre l'impatto di tali problematiche è stata prevista una gestione controllata delle chiamate API, con il trattamento degli errori e la separazione tra la logica applicativa del gestionale e i servizi esterni, così

da limitare le conseguenze di eventuali anomalie e facilitare future modifiche dell'integrazione.

Capitolo 3

Analisi dei requisiti

3.1 Tipologie di utenti

All'interno del sistema vengono individuati tre principali attori coinvolti nell'interazione con le funzionalità sviluppate:

- **Utente:** dipendente dell'azienda utilizzatrice del software, rappresenta il principale utilizzatore del gestionale. Interagisce con il sistema per consultare informazioni, ricevere segnalazioni relative a eventuali criticità e richiedere spiegazioni sulle decisioni prodotte dai diversi moduli applicativi.
- **Utente amministratore:** dipendente dell'azienda con privilegi avanzati rispetto all'utente base. Possiede tutte le funzionalità disponibili per quest'ultimo e può modificare configurazioni e impostazioni del sistema non accessibili agli utenti standard.
- **Modulo XAI:** componente software basato su chatbot che consente all'utente di interrogare il sistema e ottenere informazioni o spiegazioni relative alle decisioni prese dal gestionale. Il modulo può utilizzare meccanismi di *function calling* per recuperare dati dal database o eseguire operazioni specifiche, come la creazione di nuovi record relativi ad acquisti o vendite, previa conferma esplicita dell'utente.

3.2 Casi d'uso

I casi d'uso rappresentano una descrizione delle principali interazioni tra gli attori del sistema e le funzionalità offerte dall'applicazione. Essi permettono di definire i comportamenti attesi del sistema dal punto di vista degli utenti, descrivendo gli obiettivi che ciascun attore può raggiungere attraverso l'utilizzo delle funzionalità disponibili.

La rappresentazione dei casi d'uso consente quindi di fornire una visione ad alto livello dei requisiti funzionali del sistema, evidenziando le modalità con cui gli utenti interagiscono con il gestionale e con le nuove funzionalità introdotte.

Ogni caso d'uso è accompagnato dal relativo diagramma *UML_G*, che permette di rappresentare graficamente gli attori coinvolti e le interazioni con il sistema, rendendo più chiara e immediata la comprensione delle funzionalità descritte.

3.2.1 Struttura dei casi d'uso

Campo	Descrizione
Attori	Indica gli utenti o i componenti del sistema coinvolti nell'esecuzione del caso d'uso e che interagiscono direttamente con la funzionalità descritta.
Precondizioni	Definisce le condizioni che devono essere soddisfatte prima dell'avvio del caso d'uso affinché questo possa essere eseguito correttamente.
Postcondizioni	Descrive lo stato del sistema al termine dell'esecuzione del caso d'uso e gli eventuali risultati prodotti dall'interazione.
Scenario principale	Rappresenta il flusso di esecuzione standard del caso d'uso, descrivendo le operazioni effettuate dagli attori e le risposte fornite dal sistema.

Scenari alternativi	Descrive eventuali variazioni rispetto al flusso principale, dovute a condizioni particolari o situazioni differenti che possono verificarsi durante l'esecuzione.
Inclusioni	Indica altri casi d'uso che vengono richiamati obbligatoriamente dal caso d'uso corrente e che rappresentano funzionalità comuni riutilizzate.
Estensioni	Indica eventuali comportamenti aggiuntivi o opzionali che possono estendere il caso d'uso principale al verificarsi di specifiche condizioni.

3.3 Elenco casi d'uso

3.3.1 UC1: Configurazione del modulo XAI

- **Attori:** Utente amministratore.
- **Precondizioni:**
 1. L'utente è autenticato;
 2. L'utente possiede i permessi di amministratore.
- **Postcondizioni:** Il modulo XAI risulta configurato secondo le impostazioni definite dall'amministratore e disponibile per gli utenti autorizzati.
- **Scenario principale:**
 1. L'amministratore entra nella pagina contenente le impostazioni del modulo XAI.
 2. L'amministratore seleziona il modello *Large Language Model (LLM)_G* che vuole far utilizzare agli utenti (UC1.1).
 3. L'amministratore permette a tutte le categorie di utenti di utilizzare il modulo XAI (UC1.2).

4. L'amministratore seleziona i *tool* che vuole far utilizzare al chatbot (UC1.3).

- **Inclusioni:**

1. UC1.1: Selezione del modello LLM per il modulo XAI
2. UC1.2: Configurazione degli utenti abilitati al modulo XAI
3. UC1.3: Configurazione dei tool disponibili per il modulo XAI

3.3.1.1 UC1.1: Selezione del modello LLM per il modulo XAI

- **Attori:** Utente amministratore.

- **Precondizioni:**

1. L'amministratore si trova all'interno della pagina di impostazioni del modulo XAI.

- **Postcondizioni:** Il modulo XAI è configurato per utilizzare il modello LLM selezionato dall'amministratore.

- **Scenario principale:**

1. L'amministratore seleziona il provider da utilizzare.
2. L'amministratore seleziona il modello specifico da utilizzare.

3.3.1.2 UC1.2: Configurazione degli utenti abilitati al modulo XAI

- **Attori:** Utente amministratore.

- **Precondizioni:**

1. L'amministratore si trova all'interno della pagina di impostazioni del modulo XAI.

- **Postcondizioni:** Il modulo XAI è configurato per poter essere utilizzato dagli utenti utilizzatori del sistema.

- **Scenario principale:**

1. L'amministratore seleziona i gruppi di utenti a cui concedere l'accesso al chatbot.

3.3.1.3 UC1.3: Configurazione dei tool disponibili per il modulo XAI

- **Attori:** Utente amministratore.
- **Precondizioni:**
 1. L'amministratore si trova all'interno della pagina di impostazioni del modulo XAI.
- **Postcondizioni:** Il modulo XAI è configurato per poter utilizzare i tool selezionati dall'amministratore.
- **Scenario principale:**
 1. L'amministratore seleziona i tool che desidera rendere visibili al chatbot.

3.3.2 UC2: Richiesta di spiegazioni al modulo XAI su una decisione del sistema

- **Attori:** Utente, Modulo XAI.
- **Precondizioni:**
 1. L'utente è autenticato;
 2. L'utente possiede i permessi per poter utilizzare il chatbot;
 3. Il modulo XAI ha i tool per fare richieste abilitati.
- **Postcondizioni:** L'utente riceve una spiegazione dal modulo XAI riguardo le decisioni prese dal gestionale.
- **Scenario principale:**
 1. L'utente attiva la *sidebar* relativa al modulo XAI.
 2. L'utente accede ad una conversazione con il modulo XAI ([UC2.1](#)).

3. L'utente scrive una richiesta di spiegazioni nella chat.
4. Il chatbot effettua una chiamata a tool per poter ricevere i dati (UC2.2).
5. Il chatbot elabora i dati e li dà in output all'utente in linguaggio naturale.

- **Scenari alternativi:**

1. Il modello LLM non è raggiungibile e non è possibile generare la risposta (UC2.3).
2. Le informazioni recuperate superano il limite massimo di token elaborabili dal modello (UC2.4).
3. L'utente non dispone dei permessi necessari per visualizzare le informazioni richieste (UC2.5).

- **Inclusioni:**

1. UC2.1: Gestione della conversazione con il modulo XAI
2. UC2.2: Chiamata a tool per spiegazioni del modulo XAI

- **Estensioni:**

1. UC2.3: Modello LLM non raggiungibile
2. UC2.4: Limite token di risposta superato
3. UC2.5: Permessi non sufficienti per effettuare la richiesta

3.3.2.1 UC2.1: Gestione della conversazione con il modulo XAI

- **Attori:** Utente.

- **Precondizioni:**

1. L'utente è autenticato;
2. L'utente possiede i permessi per utilizzare il modulo XAI.

- **Postcondizioni:** L'utente accede ad una conversazione già iniziata con il modulo XAI.
- **Scenario principale:**
 1. L'utente apre la *sidebar* relativa al modulo XAI.
 2. L'utente seleziona una conversazione esistente.
 3. Il sistema carica il contesto della conversazione selezionata.
- **Scenario alternativo:**
 1. L'utente crea una nuova conversazione senza uno storico precedente.

3.3.2.2 UC2.2: Chiamata a tool per spiegazioni del modulo XAI

- **Attori:** Modulo XAI.
- **Precondizioni:**
 1. L'utente ha inviato una richiesta di spiegazioni al chatbot.
- **Postcondizioni:** Il modulo XAI possiede i dati per rispondere all'utente.
- **Scenario principale:**
 1. Il modulo XAI, in base alla richiesta, sceglie il tool di spiegazioni da chiamare per compiere il suo obiettivo.
 2. Il tool effettua le query a database necessarie per poter ricavare i dati da dare in output all'utente.
 3. Il tool formatta i dati ricevuti in un formato leggibile dall'LLM.

3.3.2.3 UC2.3: Modello LLM non raggiungibile

- **Attori:** Utente.
- **Precondizioni:**
 1. L'utente ha inviato una richiesta di spiegazioni al modulo XAI.

- **Postcondizioni:** L'applicazione ritorna un errore.
- **Scenario principale:**

1. Il modulo XAI non è raggiungibile dall'applicazione.

3.3.2.4 UC2.4: Limite token di risposta superato

- **Attori:** Utente.
- **Precondizioni:**

1. L'utente ha inviato una richiesta di spiegazioni al modulo XAI.

- **Postcondizioni:** L'applicazione ritorna un errore.
- **Scenario principale:**

1. Il sistema rileva che la quantità di dati da fornire al modello supera il limite massimo di token consentito.

3.3.2.5 UC2.5: Permessi non sufficienti per effettuare la richiesta

- **Attori:** Utente.
- **Precondizioni:**

1. L'utente ha inviato una richiesta di spiegazioni al modulo XAI.

- **Postcondizioni:** Il modulo XAI comunica all'utente che non possiede i permessi per poter ricevere la spiegazione.
- **Scenario principale:**

1. Il modulo XAI rileva che i tool che utilizza per la risposta fanno uso di modelli a cui l'utente non ha accesso.

3.3.3 UC3: Richiesta creazione nuovi record

- **Attori:** Utente, Modulo XAI.
- **Precondizioni:**
 1. L'utente è autenticato;
 2. L'utente possiede i permessi per poter utilizzare il chatbot;
 3. Il modulo XAI ha i tool per poter creare nuovi record abilitati.
- **Postcondizioni:** L'utente crea un nuovo record relativo ai dati forniti al chatbot.
- **Scenario principale:**
 1. L'utente attiva la *sidebar* relativa al modulo XAI.
 2. L'utente accede ad una conversazione con il modulo XAI ([UC2.1](#)).
 3. L'utente scrive una richiesta di creazione record nella chat.
 4. L'utente fornisce i dati necessari alla creazione del record.
 5. Il chatbot effettua una chiamata a tool per poter creare il form ([UC3.1](#)).
 6. Il chatbot richiede la conferma dei dati previa creazione del record.
- **Scenari alternativi:**
 1. Il modello LLM non è raggiungibile e non è possibile generare la risposta ([UC2.3](#)).
 2. Le informazioni recuperate superano il limite massimo di token elaborabili dal modello ([UC2.4](#)).
 3. L'utente non dispone dei permessi necessari per poter creare il record ([UC2.5](#)).
 4. I dati forniti dall'utente sono incorretti per la creazione del record ([UC3.2](#)).
- **Inclusioni:**

1. UC2.1: Gestione della conversazione con il modulo XAI
2. UC3.1: Chiamata a tool del modulo XAI

- **Estensioni:**

1. UC2.3: Modello LLM non raggiungibile
2. UC2.4: Limite token di risposta superato
3. UC2.5: Permessi non sufficienti per effettuare la richiesta
4. UC3.2: Dati forniti incorretti per la creazione del record

3.3.3.1 UC3.1: Chiamata al tool per creazione record del modulo XAI

- **Attori:** Utente.

- **Precondizioni:**

1. L'utente ha inviato una richiesta di creazione record al modulo XAI.

- **Postcondizioni:** Il modulo XAI fornisce all'utente un form compilato con tutti i dati forniti.

- **Scenario principale:**

1. Il modulo XAI richiama i tool per la creazione di un nuovo record.
2. Il modulo XAI controlla che i campi corrispondano alla richiesta dell'utente.

- **Scenario alternativo:**

1. Il modulo XAI rileva che i campi non corrispondono alle richieste dell'utente.

- **Estensioni:**

1. UC3.2: Dati forniti incorretti per la creazione del record

3.3.3.2 UC3.2: Dati forniti incorretti per la creazione del record

- **Attori:** Utente.
- **Precondizioni:**
 1. L'utente ha inviato una richiesta di creazione record al modulo XAI.
 2. L'utente ha inviato i dati per la creazione del record al modulo XAI.
- **Postcondizioni:** Il modulo XAI comunica all'utente che i dati forniti non corrispondono al record che vuole creare.
- **Scenario principale:**
 1. Il modulo XAI rileva che i campi non corrispondono alle richieste dell'utente.

3.3.4 UC4: Configurazione delle regole alert

- **Attori:** Utente.
- **Precondizioni:**
 1. L'utente è autenticato;
 2. L'utente possiede i permessi per poter utilizzare le regole alert.
- **Postcondizioni:** Le regole alert risultano attivate.
- **Scenario principale:**
 1. L'utente entra nella pagina generale degli alert.
 2. L'utente entra nella pagina delle impostazioni alert.
 3. L'utente seleziona una nuova regola da abilitare.
 4. L'utente setta i parametri della regola.

3.3.5 UC5: Visualizzazione delle notifiche alert

- **Attori:** Utente.
- **Precondizioni:**
 1. L'utente è autenticato;
 2. L'utente ha configurato le regole alert.
- **Postcondizioni:** L'utente può visualizzare la notifica *toast*.
- **Scenario principale:**
 1. Il sistema ERP rileva una criticità che soddisfa i criteri definiti da una regola alert impostata.
 2. Il sistema genera un alert relativo alla criticità rilevata.
 3. Il sistema invia una notifica *toast* all'utente.
- **Scenario alternativo:**
 1. Il sistema ERP rileva troppe violazioni della regola alert impostata.
- **Estensioni:**
 1. [UC5.1: Numero elevato di violazioni rilevate](#)

3.3.5.1 UC5.1: Numero elevato di violazioni rilevate

- **Attori:** Utente.
- **Precondizioni:**
 1. L'utente è autenticato;
 2. L'utente ha configurato le regole alert.
 3. Il sistema ha rilevato un numero elevato di violazioni della stessa regola alert.
- **Postcondizioni:** L'utente visualizza la notifica riepilogativa.

- **Scenario principale:**

1. Il sistema raggruppa le violazioni rilevate.
2. Il sistema genera una notifica riepilogativa.

3.3.6 UC6: Visualizzazione del dettaglio dell>alert

- **Attori:** Utente.

- **Precondizioni:**

1. L'utente è autenticato;
2. L'utente ha configurato le regole alert.

- **Postcondizioni:** L'utente visualizza per intero la violazione verificatasi.

- **Scenario principale:**

1. L'utente sta visualizzando la lista degli alert generati dal sistema.
2. L'utente seleziona un alert del quale vuole visualizzare i dettagli.

3.3.7 UC7: Visualizzazione del modello associato dell>alert

- **Attori:** Utente.

- **Precondizioni:**

1. L'utente è autenticato;
2. L'utente ha configurato le regole alert.
3. L'utente sta visualizzando il dettaglio di un alert.
4. Il modello associato all>alert è disponibile.

- **Postcondizioni:** L'utente visualizza il modello associato alla violazione che ha generato l>alert.

- **Scenario principale:**

1. L'utente seleziona il collegamento al modello associato all>alert.
2. Il sistema recupera il modello associato alla violazione.
3. Il sistema reindirizza l'utente alla pagina di visualizzazione del modello.

3.3.8 UC8: Richiesta al modulo XAI di una spiegazione relativa all>alert generato

- **Attori:** Utente, Modulo XAI.
- **Precondizioni:**
 1. L'utente è autenticato;
 2. L'utente ha configurato le regole alert.
 3. L'utente sta visualizzando il dettaglio di un alert.
 4. L'utente è autorizzato ad utilizzare il chatbot.
 5. La *sidebar* riguardante il chatbot è abilitata.
- **Postcondizioni:** L'utente visualizza una spiegazione relativa alla condizione che ha determinato la generazione dell>alert.
- **Scenario principale:**
 1. L'utente seleziona la funzione per richiedere una spiegazione dell>alert.
 2. Il sistema attiva la *sidebar* relativa al modulo XAI sulla conversazione precedentemente utilizzata dall'utente.
 3. Il sistema invia al modulo XAI le informazioni relative all>alert come contesto della richiesta.
 4. Il modulo XAI elabora le informazioni ricevute.
 5. Il modulo XAI fornisce una spiegazione relativa alla causa dell>alert.
 6. Il sistema mostra la spiegazione all'utente.

- **Scenari alternativi:**

1. Il modello LLM non è raggiungibile e non è possibile generare la risposta ([UC2.3](#)).
2. Le informazioni recuperate superano il limite massimo di token elaborabili dal modello ([UC2.4](#)).
3. L'utente non dispone dei permessi necessari per visualizzare le informazioni richieste ([UC2.5](#)).

- **Estensioni:**

1. [UC2.3: Modello LLM non raggiungibile](#)
2. [UC2.4: Limite token di risposta superato](#)
3. [UC2.5: Permessi non sufficienti per effettuare la richiesta](#)

3.3.9 UC9: Visualizzazione log operazioni

- **Attori:** Utente.

- **Precondizioni:**

1. L'utente è autenticato;
2. L'utente possiede i permessi per poter visualizzare il registro delle operazioni.

- **Postcondizioni:** L'utente visualizza il registro delle operazioni effettuate sui modelli del sistema.

- **Scenario principale:**

1. L'utente accede alla pagina dedicata al registro delle operazioni.
2. Il sistema recupera le operazioni registrate.
3. Il sistema mostra all'utente la lista delle operazioni effettuate sui modelli.

3.3.10 UC10: Visualizzazione dei dettagli di un'operazione

- **Attori:** Utente.
- **Precondizioni:**
 1. L'utente è autenticato;
 2. L'utente possiede i permessi necessari per visualizzare il registro delle operazioni;
 3. L'utente ha selezionato un'operazione dal registro.
- **Postcondizioni:** L'utente visualizza i dettagli relativi all'operazione selezionata.
- **Scenario principale:**
 1. L'utente seleziona un'operazione dal registro.
 2. Il sistema recupera i dettagli dell'operazione.
 3. Il sistema mostra le informazioni relative all'operazione, tra cui l'utente che l'ha effettuata, la data, il modello interessato e il tipo di operazione.
 4. Il sistema mostra i dati associati all'operazione.

3.4 Tracciamento dei requisiti

Da un'attenta analisi dei requisiti e dagli use case effettuati sul progetto è stata stilata la tabella che traccia i requisiti in rapporto agli use case.

Sono stati individuati diversi tipi di requisiti e si è quindi fatto utilizzo di un codice identificativo per distinguerli.

Il codice dei requisiti, dove ogni requisito è identificato con il carattere **R**, è così strutturato:

F: Funzionale.

Q: Di qualità.

V: Di vincolo.

N: Obbligatorio.

D: Desiderabile.

Z: Facoltativo.

Nelle tabelle [3.2](#), [3.3](#) e [3.4](#) sono riassunti i requisiti e il loro tracciamento con gli use case delineati in fase di analisi.

3.4.1 Requisti funzionali

Codice	Descrizione requisito	UC Relativo	Necessità
RFN-1	Configurazione del modulo XAI	UC1	N
RFN-2	Selezione del modello LLM per il modulo XAI	UC1.1	N
RFN-3	Configurazione degli utenti abilitati al modulo XAI	UC1.2	N
RFN-4	Configurazione dei tool disponibili per il modulo XAI	UC1.3	N
RFN-5	Richiesta di spiegazioni al modulo XAI su una decisione del sistema	UC2	N
RFN-6	Gestione della conversazione con il modulo XAI	UC2.1	N
RFN-7	Chiamata a tool per spiegazioni del modulo XAI	UC2.2	N
RFN-8	Modello LLM non raggiungibile	UC2.3	N
RFN-9	Limite token di risposta superato	UC2.4	N
RFN-10	Permessi non sufficienti per effettuare la richiesta	UC2.5	N
RFN-11	Richiesta creazione nuovi record	UC3	N
RFN-12	Chiamata al tool per creazione record del modulo XAI	UC3.1	N
RFN-13	Dati forniti incorretti per la creazione del record	UC3.2	N
RFN-14	Configurazione delle regole alert	UC4	N
RFN-15	Visualizzazione delle notifiche alert	UC5	N
RFN-16	Numero elevato di violazioni rilevate	UC5.1	N

Codice	Descrizione requisito	UC Relativo	Necessità
RFN-17	Visualizzazione del dettaglio dell'alert	UC6	N
RFN-18	Visualizzazione del modello associato dell'alert	UC7	N
RFN-19	Richiesta al modulo XAI di una spiegazione relativa all'alert generato	UC8	N
RFN-20	Visualizzazione log operazioni	UC9	N
RFN-21	Visualizzazione dei dettagli di un'operazione	UC10	N

Tabella 3.2: Tabella riguardante i requisiti funzionali del progetto

3.4.2 Requisti di qualità

Codice	Descrizione requisito	UC Relativo	Necessità
RQN-1	Il sistema deve garantire tempi di risposta adeguati nell'interazione con il modulo XAI, evitando attese non necessarie da parte dell'utente	UC1 - UC3.2, UC8	N
RQN-2	Il sistema deve garantire la sicurezza delle informazioni, impedendo agli utenti di accedere a dati per i quali non possiedono i permessi necessari	UC1 - UC10	N
RQN-3	Il sistema deve garantire la tracciabilità delle operazioni effettuate sui dati, registrando creazione, modifica ed eliminazione dei modelli	UC9 - UC10	N
RQN-4	Le spiegazioni generate dal modulo XAI devono essere comprensibili per l'utente e presentate in linguaggio naturale	UC1 - UC3.2, UC8	N
RQN-5	Il sistema deve garantire tempi di risposta adeguati durante il recupero dei dati e nella conseguente generazione e notifica degli alert	UC5, UC5.1	N
RQN-6	Il sistema deve notificare tempestivamente l'utente in caso di eccezioni nell'uso di modulo XAI e alert per ridurre le attese inutili	UC2.3 - UC2.5, UC5.1	N

Tabella 3.3: Tabella riguardante i requisiti di qualità del progetto

3.4.3 Requisti di vincolo

Codice	Descrizione requisito	UC Relativo	Necessità
RVN-1	Il sistema deve essere sviluppato in PHP	-	N
RVN-2	L'interfaccia grafica deve essere sviluppata in Filament	-	N
RVN-3	Il modulo XAI deve fornire le spiegazioni generate in lingua italiana	UC1 - UC3.2, UC8	N
RVN-4	L'integrazione con i modelli linguistici deve avvenire attraverso le API messe a disposizione dai relativi provider	UC1 - UC3.2, UC8	N

Tabella 3.4: Tabella riguardante i requisiti di vincolo del progetto

Capitolo 4

Progettazione e codifica

4.1 Tecnologie e strumenti

4.1.1 Linguaggi di programmazione

PHP è un linguaggio di programmazione general purpose particolarmente diffuso nello sviluppo di applicazioni web lato server. È stato scelto come linguaggio principale per il backend di *ErpAI* grazie alla sua maturità, alla vasta disponibilità di librerie e strumenti per lo sviluppo web e alla sua integrazione con il framework Laravel. Un ulteriore vantaggio è la possibilità di sviluppare applicazioni strutturate e facilmente manutenibili attraverso le funzionalità offerte dalle versioni moderne del linguaggio.

4.1.2 Framework e librerie

4.1.2.1 Laravel

Laravel è un framework PHP open source progettato per lo sviluppo di applicazioni web. Fornisce una struttura organizzata per la realizzazione del backend e mette a disposizione funzionalità per la gestione delle richieste HTTP, del routing, dell'accesso al database, dell'autenticazione e delle operazioni asincrone. È stato scelto per la sua ampia dotazione di funzionalità integrate, per la facilità di sviluppo e manutenzione del codice e per l'integrazione con gli altri componenti utilizzati nel progetto.

4.1.2.2 Filament

Filament è un framework per Laravel dedicato alla realizzazione di pannelli amministrativi e interfacce gestionali. Permette di costruire rapidamente risorse, tabelle, form e pagine attraverso componenti già pronti e fortemente integrati con Laravel. È stato scelto per semplificare la realizzazione dell'interfaccia gestionale di *ErpAI*, riducendo il codice necessario per implementare le funzionalità di amministrazione e mantenendo una stretta integrazione con il backend.

4.1.2.3 Livewire

Livewire è un framework full-stack per Laravel che consente di creare interfacce web dinamiche utilizzando prevalentemente codice PHP. Il suo principale vantaggio è la possibilità di aggiornare dinamicamente il DOM comunicando con il server senza dover gestire manualmente uno stato complesso lato client via JavaScript. Essendo la tecnologia su cui si basa Filament, la sua adozione garantisce la massima coerenza nello sviluppo dei componenti interattivi del gestionale.

4.1.3 Tecnologie di persistenza e gestione dei dati

4.1.3.1 MariaDB

MariaDB è un sistema di gestione di database relazionale open source, compatibile con SQL. È stato scelto come database dell'applicazione per la sua affidabilità, le buone prestazioni e l'integrazione con Laravel tramite il sistema di astrazione del database fornito dal framework. Il modello relazionale risulta inoltre particolarmente adatto alla gestione delle numerose entità e relazioni presenti in un gestionale aziendale.

4.1.3.2 Redis

Redis è un sistema di gestione dati in memoria utilizzabile come database, cache e sistema di gestione delle code. La sua caratteristica principale è la

possibilità di effettuare operazioni molto rapide grazie alla gestione dei dati prevalentemente in memoria. Nel progetto viene utilizzato per la gestione della cache e delle code, consentendo di separare le operazioni più onerose dal normale ciclo di elaborazione delle richieste.

4.1.3.3 Laravel Horizon

Laravel Horizon è un componente dell'ecosistema Laravel dedicato al monitoraggio e alla gestione delle code basate su Redis. Fornisce un'interfaccia per controllare i processi in esecuzione, le attività completate e quelle fallite, oltre a informazioni sulle prestazioni delle code. È stato scelto per semplificare la gestione delle operazioni asincrone del progetto e permettere di monitorarne lo stato direttamente dall'applicazione.

4.1.4 Ambiente di sviluppo e containerizzazione

4.1.4.1 Docker

Docker è una piattaforma di containerizzazione che permette di eseguire applicazioni e servizi all'interno di ambienti isolati chiamati container. I container consentono di distribuire insieme all'applicazione le dipendenze necessarie, riducendo le differenze tra gli ambienti di sviluppo ed esecuzione. Nel progetto viene utilizzato come base dell'ambiente di sviluppo e per l'esecuzione isolata dei diversi servizi.

4.1.4.2 DDEV

DDEV è un ambiente di sviluppo locale, basato su Docker, progettato per semplificare la configurazione di applicazioni web e dei relativi servizi. Utilizza la containerizzazione per fornire agli sviluppatori un ambiente riproducibile e isolato. È stato scelto per rendere uniforme l'ambiente di sviluppo tra i membri del team e semplificare la configurazione dei servizi necessari all'esecuzione di *ErpAI*.

4.1.5 Strumenti di qualità e testing

4.1.5.1 Pest

Pest è un framework per il testing automatizzato in ambiente PHP. Progettato con un approccio incentrato sulla sintassi fluida e l'espressività del codice, permette di scrivere unit e integration test estremamente leggibili e sintetici rispetto al classico PHPUnit. È stato scelto per verificare automaticamente il corretto funzionamento delle funzionalità del backend di *ErpAI* e prevenire l'insorgere di regressioni durante l'evoluzione del codice.

4.1.5.2 PHPStan e Larastan

PHPStan è uno strumento di analisi statica per PHP che permette di individuare potenziali errori e incongruenze nel codice senza doverlo eseguire. L'analisi viene effettuata considerando principalmente i tipi e le possibili combinazioni dei dati utilizzati dal programma. **Larastan** estende PHPStan fornendo regole e funzionalità specifiche per l'ecosistema Laravel, consentendo di analizzare correttamente costrutti e convenzioni tipiche del framework. Sono stati scelti per aumentare l'affidabilità e la qualità del codice, individuando preventivamente problematiche che potrebbero manifestarsi durante l'esecuzione dell'applicazione.

4.2 Architettura

Il progetto *ErpAI* è basato su un'architettura a componenti, nella quale il backend principale è realizzato tramite Laravel ed è responsabile della gestione delle richieste, della logica applicativa e dell'accesso ai dati. L'applicazione comunica con i servizi di persistenza e supporto, tra cui MariaDB e Redis, mentre ulteriori componenti esterni sono utilizzati per attività specifiche, quali il forecasting e l'ottimizzazione della produzione.

L'attività di stage non ha previsto una modifica dell'architettura generale dell'applicazione, ma l'estensione delle componenti già presenti attraverso

l'introduzione del modulo XAI e del sistema di alert. Le nuove funzionalità sono state integrate nell'architettura esistente, riutilizzandone i meccanismi di autenticazione, autorizzazione, accesso ai dati e gestione delle operazioni asincrone.

Per quanto riguarda l'implementazione dei moduli, è stata adottata una struttura comune basata sulla separazione tra interfaccia, implementazione condivisa e logica specifica delle singole componenti. Tale organizzazione consente di mantenere una struttura omogenea tra i diversi moduli e di favorirne l'estendibilità.

- **Contract:** interfaccia che dichiara i principali metodi che devono essere implementati dalle classi concrete. In questo modo viene definito un contratto comune per le diverse implementazioni, favorendo la compatibilità con le componenti esistenti e l'introduzione di nuove implementazioni.
- **Abstract Class:** classe astratta che implementa il **Contract** e fornisce l'implementazione dei metodi comuni alle diverse classi concrete, come i metodi getter e setter. Le sottoclassi possono quindi concentrarsi sull'implementazione della logica specifica senza dover replicare il comportamento condiviso.
- **Classi Concrete:** classi che estendono la **Abstract Class** e implementano la logica specifica della singola componente, differenziandosi in base alle funzionalità che devono fornire.

4.3 Struttura del progetto

```
erpai/  
  |- app/  
    |- Console/Commands/  
    |- Contracts/  
      |- AiChat/      - - - - - > AiToolContract.php  
      |- Alerts/      - - - - - > AlertRuleContract.php  
    |- Filament/  
      |- Clusters/  
      |- Pages/  
      |- Resources/  
    |- Jobs/          - - - - - > AlertJob.php /  
      ↪ BroadcastAlertJob.php  
    |- Models/  
    |-- Observers/    - - - - - > AuditObserver.php  
    |- Services/  
      |- AiChat/  
        |- Eval/  
        |- Prompts/  
        |- Tools/  
      |  
      |- Alerts/  
    |  
    |- bootstrap/app.php  
    |- database/  
      |- factories/  
      |- migratons/  
    |- tests/alerts/
```

Codice 4.1: Struttura del progetto

4.4 Componenti principali

4.4.1 Modulo XAI

La logica del modulo XAI è organizzata all'interno della cartella `app/Services/AiChat`. Al suo interno sono raccolte le componenti responsabili della gestione dei provider dei modelli linguistici, dei tool, dei prompt e delle funzioni di valutazione (*eval*), queste ultime descritte in dettaglio nella sezione dedicata al testing.

4.4.1.1 Tool

I tool rappresentano il punto di accesso del modello linguistico ai dati e alle funzionalità del gestionale. Attraverso di essi il modello può richiedere informazioni al sistema o eseguire operazioni, nel rispetto dei permessi configurati.

Ogni tool implementa l'interfaccia `AiToolContract`, che definisce le operazioni comuni che devono essere fornite dalle diverse implementazioni. In particolare, il contratto espone i metodi necessari al recupero dei metadati del tool, utilizzati anche per la generazione delle relative etichette nell'interfaccia utente, alla definizione dei permessi richiesti e all'esecuzione del tool.

L'interfaccia viene implementata dalla classe astratta `AbstractTool`, che fornisce il comportamento condiviso relativo alla gestione e alla verifica dei permessi. Le singole classi concrete estendono quindi `AbstractTool` e implementano la logica specifica necessaria per accedere ai dati o alle funzionalità messe a disposizione dal relativo tool.

Questa struttura permette di mantenere separata la logica comune da quella specifica dei singoli tool, facilitando l'introduzione di nuove funzionalità e mantenendo uniforme la gestione delle autorizzazioni.

4.4.1.1.1 FinderTool

Per illustrare l'implementazione dei tool viene considerato `FinderTool`, sviluppato per consentire al modello linguistico di accedere ai dati del gestionale e utilizzare informazioni reali durante la generazione delle risposte, riducendo il

rischio di produrre informazioni non presenti nel sistema.

Il tool è composto dai seguenti elementi principali:

- **shortDescription**: descrizione sintetica esposta al modello linguistico, utilizzata per consentirgli di individuare il tool più appropriato in base alla richiesta dell'utente.
- **description**: descrizione dettagliata contenente le istruzioni necessarie per guidare il modello nell'utilizzo corretto del tool.
- **parameters()**: definizione, attraverso uno schema JSON, dei parametri che il modello deve fornire per poter eseguire il tool. Nel caso di **FinderTool**, lo schema specifica le operazioni da effettuare sui dati e i modelli Eloquent sui quali eseguire le interrogazioni.
- **execute**: metodo responsabile dell'elaborazione dei parametri forniti dal modello e dell'esecuzione dell'operazione richiesta. Nel caso di **FinderTool**, i parametri definiti attraverso **parameters()** vengono elaborati per costruire ed eseguire una query sul database, consentendo di recuperare informazioni relative allo stato di una specifica entità del gestionale, come un ordine o una macchina. I risultati ottenuti vengono quindi restituiti al modello linguistico, che li utilizza per elaborare la risposta finale in linguaggio naturale.

4.4.1.2 Provider

I provider rappresentano il livello di integrazione tra il modulo XAI e i modelli linguistici. Essi gestiscono le richieste provenienti dall'applicazione e si occupano di comunicare con i servizi che espongono i modelli linguistici attraverso le relative API. In questo modo, la logica specifica necessaria per interagire con ciascun provider viene separata dal resto del modulo XAI.

Ogni provider implementa l'interfaccia **AiProviderContract**, che definisce le operazioni comuni richieste dalle diverse implementazioni. In particolare, il contratto espone i metodi necessari al recupero dei metadati dei modelli, utilizzati anche per la generazione delle relative etichette nell'interfaccia utente, nonché

alla configurazione delle modalità di *thinking* e di *streaming* e alla gestione della risposta elaborata dal modello linguistico.

L'interfaccia viene implementata dalla classe astratta **AbstractAiProvider**, che fornisce il comportamento condiviso tra le diverse implementazioni, occupandosi della gestione dei parametri comuni e dell'interazione con i tool. Le singole classi concrete estendono quindi **AbstractAiProvider** e implementano la logica specifica richiesta dai diversi provider e modelli.

Tra le funzionalità gestite dalle implementazioni concrete rientrano la gestione delle chiamate ai tool, il rispetto dei limiti imposti dal modello relativamente ai tempi di elaborazione e al numero di token in ingresso e in uscita, la gestione della modalità di *thinking*, quando supportata dal modello, e la gestione dello *streaming*. Quest'ultima permette di trasmettere la risposta progressivamente, suddividendola in più porzioni, riducendo così il tempo percepito dall'utente prima della visualizzazione del contenuto.

Vi è infine il metodo **reply**, responsabile della gestione dell'interazione con il modello linguistico. Esso si occupa del caricamento dei tool disponibili, della costruzione della conversazione e del relativo contesto, nonché della configurazione dei parametri relativi ai limiti e alle modalità di risposta. Una volta predisposti i dati necessari, il metodo effettua la chiamata alle API del provider e restituisce la risposta elaborata dal modello.

4.4.1.2.1 RunpodOllamaProvider

Per illustrare l'implementazione dei provider viene considerato **RunpodOllamaProvider**, sviluppato per consentire al modulo XAI di utilizzare un modello linguistico eseguito tramite Ollama¹ su un'istanza Runpod². Durante lo sviluppo, questo provider è stato quello principalmente utilizzato, grazie alla disponibilità del modello open source Qwen3.6:35B³ e alle prestazioni di risposta ottenute nell'ambiente di esecuzione utilizzato.

¹ Ollama. URL: <https://ollama.com/>

² Runpod. URL: <https://www.runpod.io/>

³ Qwen3.6:35b. URL: <https://huggingface.co/Qwen/Qwen3.6-35B-A3B>

L'implementazione di `RunpodOllamaProvider` è volutamente ridotta, poiché la maggior parte della logica comune ai diversi provider è implementata all'interno di `AbstractAiProvider`. La classe concreta si occupa principalmente di fornire i parametri specifici necessari per l'accesso al servizio, quali l'URL dell'API, la chiave di autenticazione e i limiti relativi al numero di token supportati dal modello.

4.4.1.3 Services

I `Services` costituiscono il livello di coordinamento tra l'interfaccia utente e le componenti del modulo XAI. In questa componente è stata implementata la classe `AiChatService`, responsabile della gestione delle principali operazioni necessarie per l'interazione con il chatbot.

4.4.1.3.1 AiChatService

La classe `AiChatService` coordina l'intero ciclo di utilizzo del modulo XAI. In particolare, gestisce il contesto associato alla richiesta, composto dalle informazioni provenienti dalla pagina del gestionale e dalla conversazione corrente. Il contesto permette al modello linguistico di interpretare correttamente anche richieste che, considerate singolarmente, potrebbero risultare incomplete.

La classe si occupa inoltre della verifica dei permessi necessari per l'accesso alla *sidebar* del chatbot e della gestione delle conversazioni, inclusa la creazione di nuove conversazioni e il recupero di quelle precedentemente avviate. Un'ulteriore responsabilità riguarda la registrazione e la gestione dei tool disponibili per il provider selezionato.

Infine, `AiChatService` espone il metodo `sendMessage`, che coordina l'invio della richiesta al modello linguistico. Il metodo raccoglie il contesto della conversazione, il modello configurato, i tool disponibili e il messaggio dell'utente, delegando quindi al provider la gestione della comunicazione con il modello e restituendo il risultato alla componente chiamante.

4.4.2 Sistema di generazione alert

La logica del sistema di generazione alert è organizzato in varie componenti.

4.4.2.1 Rules

Le rules permettono di effettuare ricerche al database periodicamente per ricercare delle violazioni delle regole settate. La maggior parte delle regole non sono generali ma permettono all'utente di settare parametri opzionali per personalizzare le notifiche che questo ha necessità ricevere.

Queste, come i tool forniscono un interfaccia base `AlertRuleContract` che fornisce il contratto per tutte le sottoclassi figlie, una classe astratta `AlertRule` che fornisce un implementazione per i metodi comuni (in questo caso i metodi che permettono alla rule di accedere ai parametri settati dall'utente e che gestiscono i permessi di utilizzo).

Infine si ha l'implementazione vera e propria della regola, contenente i metadati utilizzati per la UI e i metodi `check`, utilizzato per effettuare le query a database per trovare le violazioni della stessa e `toAlert`, la quale formatta le informazioni ricevute e ritorna un *Data Transfer Object (DTO)*_G che verrà poi adattato in un alert.

Le regole alert implementate rispondono a varie esigenze degli utenti finali di *ErpAI* e sono state implementate per poter rispondere tempestivamente a queste esigenze. Alcuni esempi comprendono *LateBuyOrdersRule* e *LateInvoicesRule* le quali ritornano notifiche riguardanti gli ordini (di acquisto o di vendita rispettivamente) in ritardo rispetto alle aspettative, *SchedulingErrorsRule* la quale segnala errori e discrepanze nei processi di scheduling della produzione, oppure *PausedMachineRule* la quale ritorna le macchine che non stanno eseguendo alcuna lavorazione, compreso di tempistiche e causa della pausa.

4.4.3 Sistema di generazione degli alert

La logica del sistema di generazione degli alert è organizzata in diverse componenti, ciascuna con una responsabilità specifica. Il sistema è basato su un in-

sieme di regole configurabili che verificano periodicamente lo stato del gestionale e individuano eventuali condizioni che richiedono l'intervento dell'utente.

4.4.3.1 Rules

Le *rules* rappresentano le componenti responsabili della verifica delle condizioni che possono generare un alert. Esse effettuano interrogazioni al database per individuare eventuali violazioni delle regole configurate dall'utente. La maggior parte delle regole prevede inoltre parametri configurabili, che permettono di personalizzare le condizioni di attivazione degli alert in base alle esigenze dell'utente.

Analogamente ai tool del modulo XAI, le regole implementano l'interfaccia **AlertRuleContract**, che definisce le operazioni comuni che devono essere fornite dalle diverse implementazioni. Il contratto viene implementato dalla classe astratta **AlertRule**, che fornisce il comportamento condiviso tra le diverse regole, in particolare per quanto riguarda l'accesso ai parametri configurati dall'utente e la gestione dei permessi di utilizzo.

Le singole regole concrete estendono quindi **AlertRule** e implementano la logica specifica necessaria per individuare le relative violazioni. Ogni regola definisce inoltre i metadati utilizzati dall'interfaccia utente e implementa principalmente due metodi:

- **check**: esegue le verifiche necessarie, effettuando le interrogazioni al database per individuare le condizioni che soddisfano i criteri della regola;
- **toAlert**: elabora le informazioni relative alla violazione individuata e le converte in un *Data Transfer Object (DTO)_G*, che verrà successivamente utilizzato per la generazione dell'alert.

Le regole implementate rispondono a diverse esigenze operative degli utenti finali di *ErpAI*, con l'obiettivo di individuare tempestivamente situazioni che possono richiedere un intervento. Tra gli esempi implementati sono presenti *LateBuyOrdersRule* e *LateInvoicesRule*, che individuano rispettivamente ordini di acquisto e fatture in ritardo rispetto alle condizioni previste, *SchedulingErrorsRule* e *ForecastErrorsRule*, che segnalano errori o discrepanze nei processi di scheduling

della produzione e nelle predizioni delle vendite, e *PausedMachineRule*, che individua le macchine che non stanno eseguendo alcuna lavorazione, fornendo anche informazioni relative alla durata e alla causa della pausa.

4.4.3.2 Jobs

I *job* sono le componenti responsabili dell'esecuzione asincrona delle operazioni necessarie alla generazione e alla distribuzione degli alert. Essi vengono avviati dal sistema di pianificazione ed eseguono i controlli definiti dalle regole, la persistenza degli alert e la successiva gestione delle notifiche.

4.4.3.2.1 AlertJob

La classe **AlertJob** costituisce il principale punto di ingresso del motore di generazione degli alert. Essa espone i metodi **handle**, che contiene la logica principale di elaborazione, e **tooManyAlerts**, utilizzato per gestire il caso in cui il numero di violazioni individuate superi la soglia massima configurata.

Il metodo **handle** esegue le *rules* abilitate dall'utente e raccoglie i risultati delle verifiche. Gli alert individuati vengono quindi persistiti nel database. Successivamente viene verificato il numero di alert che devono essere notificati all'utente. Se tale numero è inferiore alla soglia massima prevista, vengono create le relative notifiche e viene effettuato il *dispatch* di un **BroadcastAlertJob** per la loro distribuzione all'interfaccia utente.

L'esecuzione della distribuzione delle notifiche tramite una coda asincrona permette di separare la generazione degli alert dalla loro visualizzazione, evitando che un numero elevato di notifiche rallenti l'elaborazione principale. La gestione delle code è affidata a Laravel Horizon, che utilizza Redis come sistema di coda. Se invece il numero di alert supera la soglia configurata, viene richiamato il metodo **tooManyAlerts**. In questo caso il sistema evita la generazione di un numero eccessivo di notifiche e produce una notifica specifica che informa l'utente della presenza di un numero elevato di violazioni.

4.4.3.2 BroadcastAlertJob

La classe `BroadcastAlertJob` si occupa della distribuzione degli alert verso l'interfaccia utente. Il job crea la *Notification Filament* corrispondente all'alert e ne gestisce il *broadcasting*, consentendo alla notifica di essere recapitata all'utente destinatario.

L'esecuzione di questa operazione tramite un job separato permette di mantenere indipendente la generazione degli alert dalla loro distribuzione, sfruttando il sistema di code asincrone fornito da Laravel Horizon e Redis.

4.4.3.3 Command

Il motore di generazione degli alert viene avviato tramite un comando Artisan, eseguibile manualmente da terminale mediante `php artisan alert` oppure automaticamente attraverso il sistema di scheduling di Laravel. In ambiente di produzione, il comando viene registrato nello scheduler dell'applicazione, definito in `bootstrap/app.php`, che ne pianifica l'esecuzione secondo intervalli temporali prestabiliti.

4.4.3.3.1 AlertCommand

La classe `AlertCommand` svolge esclusivamente il ruolo di punto di ingresso del motore di generazione degli alert e non contiene direttamente la logica necessaria all'elaborazione delle regole.

Il metodo `handle`, unica operazione implementata dal comando, individua gli utenti che dispongono di almeno una regola alert abilitata e procede al *dispatch* di un `AlertJob` per ciascuno di essi. L'elaborazione vera e propria viene quindi delegata ai job, permettendo al comando di terminare rapidamente e lasciando l'esecuzione dei controlli al sistema di code asincrone.

4.4.3.4 Resources e Pages

Per la realizzazione dell'interfaccia relativa al sistema di alert sono state sviluppate, utilizzando le funzionalità messe a disposizione da Filament, una

Resource e due *Pages*. Queste componenti sono implementate all'interno dell'applicazione Laravel e permettono di integrare direttamente la logica del sistema di alert con gli elementi dell'interfaccia utente.

- **AlertResource:** risorsa che gestisce la visualizzazione della lista degli alert rilevati;
- **ViewAlert:** pagina dedicata alla visualizzazione del dettaglio di un singolo alert. Fornisce inoltre le funzionalità necessarie per richiedere una spiegazione dell'evento tramite il modulo XAI e per raggiungere la risorsa del gestionale associata all'evento che ha generato l'alert;
- **AlertSettings:** pagina dedicata alla configurazione del sistema di alert, attraverso la quale l'utente può aggiungere, configurare e rimuovere le regole alert disponibili.

4.4.4 Sistema di logging delle operazioni

Il sistema di logging delle operazioni si basa sull'utilizzo di una classe *Observer*, incaricata di intercettare le principali operazioni effettuate sui modelli Eloquent presenti nel sistema. In particolare, vengono registrati a database gli eventi relativi alla creazione, alla modifica e all'eliminazione dei record, permettendo di mantenere uno storico delle operazioni effettuate dagli utenti.

4.4.4.1 Observers

4.4.4.1.1 AuditObserver

AuditObserver è la classe *Observer* responsabile della registrazione delle operazioni effettuate sui modelli Eloquent. Essa implementa i metodi `created()`, `updated()` e `deleted()`, che vengono richiamati automaticamente dal framework in corrispondenza delle relative operazioni. I metodi dell'*Observer* delegano quindi la costruzione del record di audit a un metodo comune, `base`, che pre-dispone un'istanza di **AuditLog** contenente le informazioni necessarie alla registrazione dell'operazione. Il record viene successivamente completato con i dati specifici dell'evento e persistito nel database.

4.4.4.2 Resources e Pages

Per la realizzazione dell'interfaccia relativa al sistema di logging sono state sviluppate una *Resource* e una *Page*, quest'ultima integrata con una vista *Blade* dedicata alla visualizzazione dei dettagli.

- **AuditLogResource:** risorsa che gestisce la visualizzazione dell'elenco delle operazioni registrate nel sistema;
- **ViewAuditLogPage:** pagina dedicata alla visualizzazione del dettaglio di un singolo log. A differenza della pagina **ViewAlert**, non utilizza direttamente i componenti di visualizzazione dei record forniti da Filament, ma passa i dati alla relativa vista *Blade* per la loro elaborazione;
- **view-audit-log.blade:** vista *Blade* responsabile della visualizzazione delle modifiche effettuate sul modello associato al log. L'utente può scegliere se visualizzare l'intero storico dell'istanza oppure limitarsi all'operazione selezionata. È inoltre possibile visualizzare tutte le proprietà del modello oppure esclusivamente quelle interessate dalla modifica. Per ogni operazione vengono infine riportati la data e l'utente responsabile dell'azione.

4.5 Design Pattern utilizzati

4.5.1 Observer Pattern

La classe **AuditObserver** implementa l'*Observer Pattern* e viene registrata all'interno di **AppServiceProvider**, classe responsabile della registrazione e configurazione dei servizi dell'applicazione. Per ogni modello Eloquent soggetto a tracciamento viene effettuata la registrazione dell'Observer tramite il metodo `$model::observe(AuditObserver::class)`.

A seguito della registrazione, Laravel provvede a notificare automaticamente **AuditObserver** quando sul modello si verificano gli eventi **created**, **updated** o **deleted**. L'Observer intercetta quindi tali eventi e provvede alla creazione

del relativo record di `AuditLog`, permettendo di mantenere una registrazione centralizzata delle operazioni effettuate sui modelli del sistema.

4.5.2 Strategy Pattern

Il *Strategy Pattern* viene utilizzato per rendere intercambiabili le implementazioni dei provider LLM e delle regole alert. Entrambe le componenti sono definite attraverso un'interfaccia comune, che permette al resto dell'applicazione di interagire con le diverse implementazioni senza dipendere direttamente dalle loro caratteristiche specifiche.

Nel modulo `XAI`, `AiProviderContract` definisce le operazioni comuni che devono essere fornite dai diversi provider. Le classi concrete, come `RunpodOllamaProvider`, implementano tale contratto fornendo le modalità specifiche di comunicazione con il servizio LLM. In questo modo il provider utilizzato può essere sostituito senza modificare la logica del servizio che gestisce la conversazione.

Un approccio analogo viene adottato per il sistema di alert tramite `AlertRuleContract`. Ogni regola concreta implementa lo stesso contratto, fornendo una propria strategia per individuare le violazioni e trasformarle in alert. Il motore di generazione può quindi eseguire le diverse regole senza dover conoscere nel dettaglio la logica specifica di ciascuna di esse.

L'utilizzo di questo pattern permette di aggiungere nuovi provider o nuove regole alert mantenendo separata la loro logica specifica dal resto dell'applicazione e riducendo le modifiche necessarie alle componenti già esistenti.

4.5.3 Command Pattern

Il sistema di generazione degli alert utilizza un comando dedicato, `AlertCommand`, che costituisce il punto di ingresso per l'avvio dell'elaborazione. Il comando incapsula l'operazione di avvio del controllo delle regole alert e ne separa l'esecuzione dalla logica che implementa effettivamente il processo.

Il metodo `handle()` di `AlertCommand` individua gli utenti che possiedono regole alert abilitate e provvede al *dispatch* di un `AlertJob` per ciascuno di essi. In questo modo il comando non contiene direttamente la logica di verifica delle

regole, ma si limita ad avviare il processo, demandandone l'esecuzione ai job. Il comando può essere eseguito manualmente tramite Artisan oppure automaticamente attraverso lo scheduler dell'applicazione. Questa separazione permette di mantenere indipendenti il meccanismo di avvio e la logica di elaborazione degli alert.

4.5.4 Template Method Pattern

Il *Template Method Pattern* viene utilizzato nella gestione dei provider del modulo XAI. La classe astratta **AbstractAiProvider** definisce il comportamento comune necessario per elaborare una richiesta al modello linguistico, centralizzando le operazioni condivise tra i diversi provider.

In particolare, il metodo `reply()` gestisce il flusso generale dell'elaborazione, occupandosi della costruzione della conversazione, del caricamento dei tool, della gestione del contesto e dei parametri relativi alla richiesta. Le implementazioni concrete dei provider estendono quindi la classe astratta e forniscono le informazioni e i comportamenti specifici necessari per comunicare con il relativo servizio.

Lo stesso approccio viene utilizzato anche nella struttura dei tool e delle regole alert, attraverso le rispettive classi astratte **AbstractTool** e **AlertRule**, che centralizzano la gestione delle funzionalità comuni lasciando alle classi concrete l'implementazione della logica specifica.

Bibliografia

Sitografia

Gitea. URL: <https://about.gitea.com/> (cit. a p. 7).

Ollama. URL: <https://ollama.com/> (cit. a p. 44).

Plane. URL: <https://plane.so/> (cit. a p. 6).

Qwen3.6:35b. URL: <https://huggingface.co/Qwen/Qwen3.6-35B-A3B> (cit. a p. 44).

Runpod. URL: <https://www.runpod.io/> (cit. a p. 44).

Scrum. URL: <https://scrumguides.org/scrum-guide.html> (cit. a p. 5).

Sito ufficiale di Spazio Dev S.r.l. URL: <https://www.spaziodev.eu>.